Banking System

Java OOP Console Application

Problem Statement

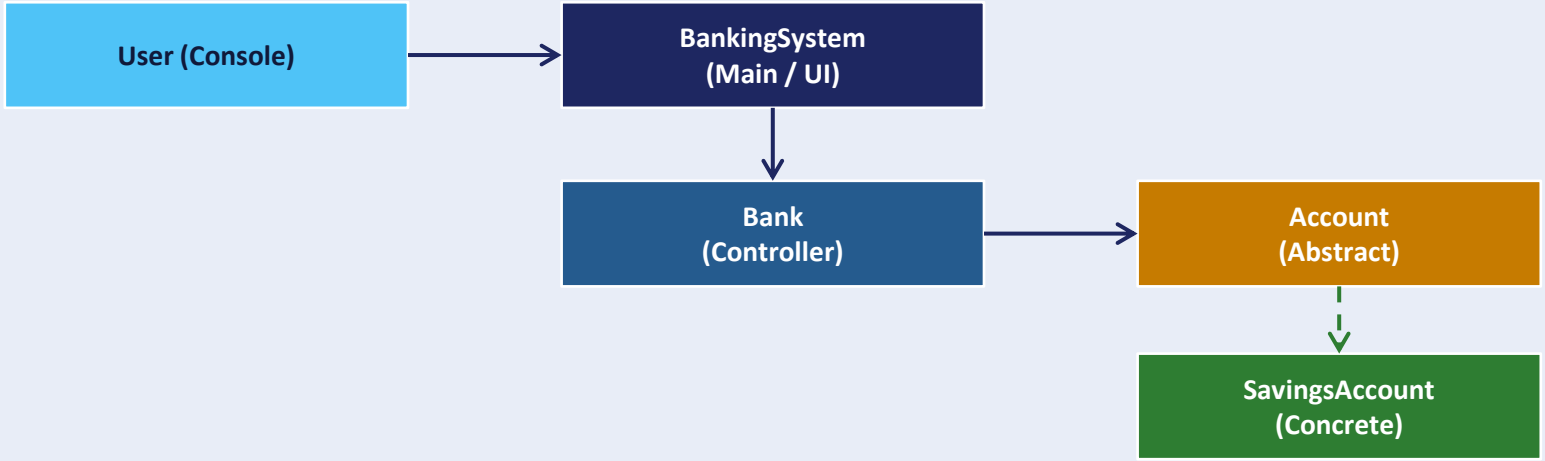
Manual banking operations are error-prone and inefficient. There is no unified system to manage accounts, deposits, withdrawals, and transfers reliably.

Target User: Banking customers & developers learning OOP design

5 min

System Design

High-Level Architecture



Key Classes & Responsibilities

Account: Abstract base — holds balance, history; declares deposit/withdraw

Bank: Manages account map, creates accounts, orchestrates transfers

SavingsAccount: Concrete impl — validates and executes deposit & withdraw

BankingSystem: Entry point — scanner-driven menu loop, dispatches to Bank

OOP Concepts Used

◈ Abstraction

- Account is declared abstract
- deposit() and withdraw() are abstract methods
- Hides implementation details from users

⬆ Inheritance

- SavingsAccount extends Account
- Inherits: balance, history, checkBalance()
- Reuses constructor via super()

⟳ Polymorphism

- @Override on deposit() & withdraw()
- Bank stores Map<Integer, Account>
- Calls deposit/withdraw via Account ref

⬡ Encapsulation

- balance & accounts are private/protected
- Public API: deposit(), withdraw(), transfer()
- Internal state not directly modifiable

Design Pattern: Template Method Pattern — Account defines the algorithm skeleton; SavingsAccount provides concrete implementation

Technical Highlight

Design Decision: Exception-Driven Validation

```
// SavingsAccount.withdraw()
if (amount > balance) {
  throw new RuntimeException(
    "Insufficient Balance!"); }
balance -= amount;
history.add("Withdrawn: " + amt);

// Bank.transfer()
Account sender = getAccount(from);
Account receiver = getAccount(to);
if (sender.balance < amount)
  throw new RuntimeException(...);
sender.balance -= amount;
receiver.balance += amount;
```

Why This Approach?

Separation of Concerns
Validation logic lives in the class, not the menu controller

Consistent Error Handling
try-catch wraps all user interactions in main loop

Polymorphic Dispatch
Bank calls withdraw() via Account ref — works for any future account type

Audit Trail
Every valid transaction is recorded in transactionHistory list

Trade-off Noted
RuntimeException is unchecked — could be improved with custom checked exceptions

Live Demo

01 Happy Path

1. Create two accounts (Alice & Bob)
2. Deposit ₹5000 into Alice's account
3. Transfer ₹2000 from Alice to Bob
4. Check balances of both accounts

02 Meaningful Scenario

1. Withdraw ₹1000 from Bob's account
2. Print transaction history for Bob
3. Show history reflects all operations
4. Confirm audit trail is accurate

03 Error Handling

1. Attempt withdraw > balance → 'Insufficient Balance!'
2. Deposit a negative amount → 'Invalid deposit amount!'
3. Access non-existent account → 'Account not found!'
4. Show system continues gracefully

💡 Tip: Compile with `javac BankingSystem.java` then run `java BankingSystem`